OpenDeepResearcher

Agentic LLM Research Framework

Milestone 1 – Foundation Setup

Submitted By:
Mahesh Reddy

Program:
Infosys Springboard Internship 6.0

Guide/Mentor:
Deepanshu Choudhary

Submission Date:
01 March 2026

1. Executive Summary

Milestone 1 focused on establishing the technical and architectural foundation of the OpenDeepResearcher system. The objective was to configure the development environment, design the agentic architecture, integrate a local Large Language Model (LLM), connect external search APIs, and validate a stub research workflow.

By the end of this milestone:

- The Python development environment was fully configured.
- Project architecture was clearly defined and documented.
- A local instruction-tuned LLM was integrated via API.
- Tavily web search API was successfully configured.
- A stubbed research flow was executed end-to-end.

2. Project Objective

The goal of OpenDeepResearcher is to build an AI-powered research assistant capable of autonomously:

- Planning research tasks
- Retrieving relevant information from the web
- Synthesizing structured responses
- Generating coherent research reports

The system follows a multi-agent workflow to simulate how a human researcher plans, gathers, and writes information.

3. Development Environment Setup

3.1 Repository Structure

```
opendeepresearcher/
 ├agents/
      ├── planner.py
      ├── searcher.py
      ├── writer.py
      ├── main.py
 ├── requirements.txt
 ├── .env
 └── README.md
```

3.2 Virtual Environment Setup

A virtual environment was created to isolate dependencies and prevent version conflicts.

Commands Used

```
python -m venv venv

venv\Scripts\activate

pip install -r requirements.txt
```

Reasoning

- Ensures dependency isolation
- Avoids global package conflicts
- Makes the project portable and reproducible

4. Project Architecture Design

4.1 Architectural Overview

The system follows a modular multi-agent architecture:

1. User enters research query
2. Planner Agent breaks query into sub-questions
3. Searcher Agent retrieves relevant information
4. Writer Agent synthesizes structured output
5. Final response returned to user

This separation follows the **Single Responsibility Principle**, improving:

- Maintainability
- Scalability Debugging efficiency
- Modular upgrades

4.2 Architecture Flow Diagram

The execution flow is controlled using a directed graph model to ensure deterministic transitions between agents.

4.3 Component Responsibilities

Planner Agent

- Receives main research topic
- Breaks it into logical sub-questions
- Creates structured research roadmap
- Uses LLM reasoning capability

Why LLM is used here:
Planning requires semantic understanding and decomposition, which is a reasoning-heavy task.

Searcher Agent

- Queries Tavily API
- Retrieves relevant content
- Returns raw search results (no summarization)

Why it does NOT summarize:
Summarization at this stage may remove critical context. The Writer Agent must see full content to ensure high-quality synthesis.

Writer Agent

- Receives sub-question + raw search results
- Synthesizes structured response
- Generates coherent summary using LLM

Why it needs both:
The sub-question provides context. The raw content provides factual grounding. Without both, hallucination risk increases.

5. Local LLM Integration

5.1 Model Used

Instruction-tuned 7B parameter model running locally (eg.Qwen2.5-7b-instruct)

5.2 Integration Method

- Model served via local runtime
- OpenAI-compatible API endpoint
- Accessed using LangChain interface

6. External Service Integration

6.1 Tavily API Setup

Steps:

1. Created Tavily account
2. Generated API key
3. Stored key securely in .env file
4. Loaded using python-dotenv

Issue Encountered

Error: MissingAPIKeyError

Cause:
Environment variable not properly loaded.

Resolution:

- Verified .env file format
- Ensured correct variable naming
- Restarted environment session

After correction, search requests executed successfully.

7. Stubbed Research Flow Validation

A basic test was performed:

Input:

"What are the impacts of AI in education?"

Output Behavior:

- Planner returned structured sub-questions
- Searcher retrieved relevant web results
- Writer generated summary paragraphs

This confirms:

- Agent pipeline functions correctly
- API integrations work
- LLM produces coherent outputs

Quality optimization is deferred to Milestone 2.